Entrega 1 — ALU Takum e Instruções Vetoriais Básicas

Projeto Integrador AOC 2026/02 — Formato Takum na Extensão Vetorial RISC-V

Heloisa Silva Rosa Samantha Kelly de Souza Sena
Vitor Hugo Lacerda Lana

Arquitetura e Organização de Computadores — DELT/UFMG

Outubro de 2026

Resumo. Relatório parcial das semanas 3–4. Implementamos em Verilog o decodificador, o codificador e a ALU (soma, subtração e multiplicação) do formato *takum* linear para T8, T16 e T32, além de uma unidade vetorial com as instruções `vtadd.vv`, `vtsub.vv` e `vtmul.vv`. Todos os testes passam. Este documento não é o artigo final.

1 O que foi feito

Item	Situação
Decodificador e codificador takum (T8, T16, T32)	pronto, testado
ALU: soma, subtração e multiplicação	pronta, testada
Unidade vetorial `vtadd.vv`, `vtsub.vv`, `vtmul.vv`	pronta, testada
`vtdot.vv`, `vncvt.t.t`, integração no modelo RVV	Fase 2
Benchmarks de álgebra linear esparsa e erro relativo	Fase 3

O código está no repositório: RTL em `rtl/`, testes em `tb/`. Para rodar: `make test`.

2 Formato takum

Usamos o takum linear de Hunhold [1]. Uma palavra de n bits tem os campos S (sinal), D (direção), R (regime, 3 bits), C (característica, r bits) e F (fração):

$$r = \begin{cases} R & D = 1 \\ 7 - R & D = 0 \end{cases} \qquad c = \begin{cases} 2^r - 1 + C & D = 1 \\ -2^{r+1} + 1 + C & D = 0 \end{cases} \qquad x = (1 + f) \cdot 2^c \ \ (x > 0).$$

Negativos são o complemento de dois da palavra. $00\cdots0$ é zero e $10\cdots0$ é NaR. O que importa para o hardware: os campos S, D, R, C cabem sempre nos 12 primeiros bits, para qualquer n.

3 Arquitetura

Toda operação passa por três blocos combinacionais (Figura 1):

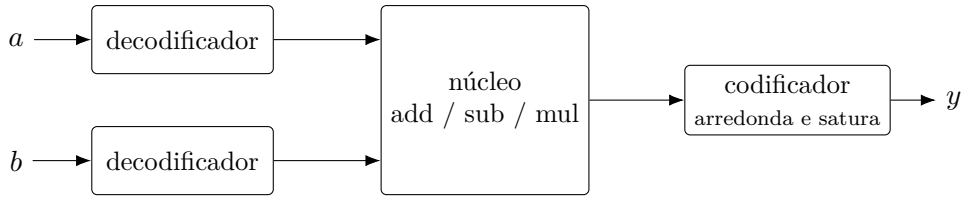


Figura 1: Caminho de dados da `takum_alu`.

- **Decodificador** (`takum_decoder`): converte a palavra em sinal, expoente (9 bits) e fração. Nega a palavra se for negativa e lê os 12 bits de cima com um bloco fixo (`takum_hdr_dec`), o mesmo para T8, T16 e T32.
- **Núcleo** (`takum_alu`): *multiplicação* soma os expoentes e multiplica os significandos; *soma/subtração* alinha o menor operando, soma ou subtrai e normaliza. A subtração é $a + (-b)$. Os bits descartados viram *guard*, *round* e *sticky*.
- **Codificador** (`takum_encoder`): monta a palavra de volta, arredonda para o mais próximo (empate para o par) e satura: um resultado não nulo nunca vira 0 nem NaR.

O arredondamento é feito direto na palavra, porque o takum é monótono: somar 1 ao padrão de bits leva ao próximo valor representável.

Código 1: `rtl/takum_encoder.v`, linhas 57–62: arredondamento

```
57    // arredondamento (RNE na palavra) ---------------------------------
58    wire [N-1:0] w     = full[N+8:9];                    // N bits de cima
59    wire         guard = full[8];
60    wire         stk   = (|full[7:0]) | sticky;
61    wire         rnd   = guard & (stk | w[0]);
62    wire [N-1:0] w_rnd = w + {{(N-1){1'b0}}, rnd};
```

Casos especiais. NaR em qualquer operando dá NaR; $x - x = +0$; $0 \times x = 0$.

Unidade vetorial (`takum_vec_alu`). Calcula `vd[i] = vs2[i] op vs1[i]` para `i < vl` e mantém `vd[i]` nos demais elementos. A largura do elemento (`sew` = 8, 16 ou 32) escolhe qual fileira de ALUs vai para a saída. Máscaras e CSRs ficam para a Fase 2.

4 Verificação

A ALU é comparada com um oráculo em Python (`tb/takum_oracle.py`) que calcula o resultado exato com frações e não compartilha código com o RTL.

Tabela 1: Resultado de `make test`.

Teste	Casos	Resultado
Decodificador (T8/T16 completos, T32 amostrado)	265 800	PASS
Ida e volta `encoder(decoder(t)) == t`	265 792	PASS
Codificador (arredondamento, empates, saturação)	323 816	PASS
ALU T8 (todos os pares, 3 operações)	196 608	PASS
ALU T16 / T32 (bordas e aleatórios)	109 923 cada	PASS
Unidade vetorial SEW 8/16/32	5 000 cada	PASS

Para conferir que os testes pegam erros, inserimos cinco defeitos de propósito na ALU (por exemplo, esquecer o *sticky* ou não normalizar o produto). Todos foram detectados.

A Figura 2 mostra o codificador arredondando, gerada com `make waves`.

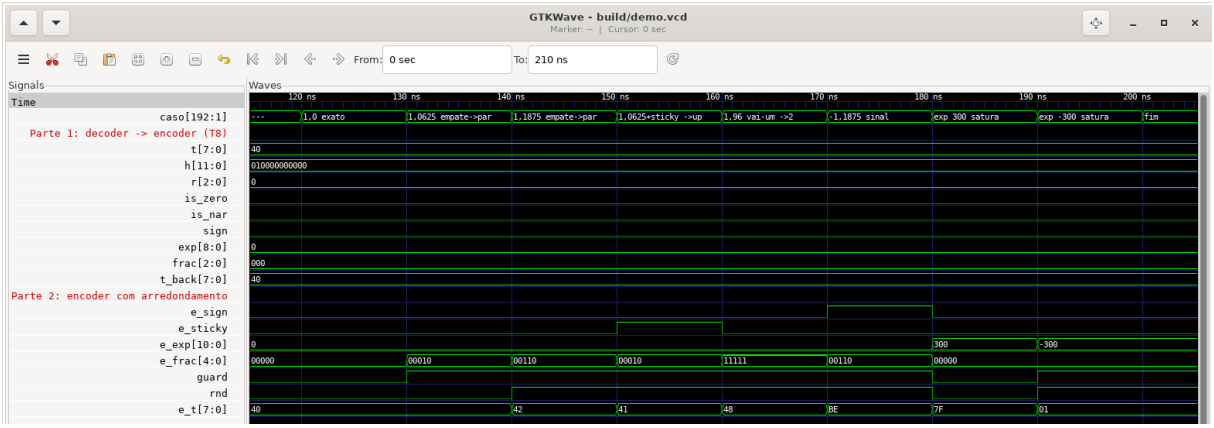


Figura 2: Codificador T8 no GTKWave: `rnd` indica quando a palavra é incrementada. Em 1,0625 o empate fica no par (`40`); em 1,1875 sobe (`42`); com expoente ± 300 satura (`7F`, `01`).

5 Custo de hardware

Síntese genérica com Yosys (`make synth`). Serve para comparar tamanhos, não como área real.

LUT6	T8	T16	T32
`takum_decoder`	46	89	164
`takum_encoder`	69	112	209
`takum_alu`	327	912	2 838

O bloco do cabeçalho custa 41 LUT6 em qualquer tamanho. Em T32 o multiplicador domina a ALU.

6 Próximos passos

- Fase 2: `vtdot.vv`, `vncvt.t.t` e integração ao modelo RVV.
- Fase 3: multiplicação matriz esparsa $\times$ vetor e erro relativo contra IEEE 754 e Float128.
- Comparar a área com uma ALU IEEE 754 equivalente.

Referências

[1] L. Hunhold, "Beating posits at their own game: Takum arithmetic," in *CoNGA 2024*, LNCS 14666, Springer, 2024. arXiv:2404.18603.

[2] L. Hunhold, "Streamlining SIMD ISA extensions with takum arithmetic: A case study on Intel AVX10.2," *MOCAST*, 2025. arXiv:2503.14067.

A Código RTL

Código 2: `rtl/takum_hdr_dec.v` — cabeçalho de 12 bits

```
// takum_hdr_dec.v -- Decodificador do cabeçalho de 12 bits de um takum linear

`timescale 1ns/1ps

module takum_hdr_dec (
    /* verilator lint_off UNUSEDSIGNAL */  // h[11] (S) é ignorado de propósito
    input  wire [11:0]       h,
    /* verilator lint_on UNUSEDSIGNAL */
    output wire [2:0]        r,      // nº de bits de característica (0..7)
    output wire signed [8:0] exp     // expoente e = c, de -255 a +254
);
    wire       d     = h[10];
    wire [2:0] reg_r = h[9:7];

    // r = D ? R : 7 - R   (7 - R == ~R em 3 bits)
    assign r = d ? reg_r : ~reg_r;

    // C: r bits alinhados à direita (0 .. 2^r - 1)
    wire [6:0] c_bits = h[6:0] >> (3'd7 - r);

    // máscara com r bits em 1, e o "1 implícito" na posição r
    wire [7:0] mask = (8'd1 << r) - 8'd1;
    wire [7:0] one  =  8'd1 << r;

    // {1, C} e {1, ~C}  (8 bits, valores 1 .. 255)
    wire [7:0] a_pos = one | {1'b0, c_bits};
    wire [7:0] a_neg = one | (~{1'b0, c_bits} & mask);

    assign exp = d ? $signed({1'b0, a_pos}) - 9'sd1
                   : -$signed({1'b0, a_neg});
endmodule
```

Código 3: `rtl/takum_decoder.v` — decodificador

```
// takum_decoder.v -- Decodificador de takum linear de N bits

`timescale 1ns/1ps

module takum_decoder #(
    parameter N = 16
) (
    input  wire [N-1:0]      t,
    output wire              is_zero,
    output wire              is_nar,
    output wire              sign,
    output wire signed [8:0] exp,
    output wire [N-6:0]      frac
);
    // ---- casos especiais ------------------------------------------------
    wire rest_zero = ~|t[N-2:0];
    assign is_zero = ~t[N-1] & rest_zero;
    assign is_nar  =  t[N-1] & rest_zero;
    wire   special = rest_zero;              // zero ou NaR

    // ---- negação condicional --------------------------------------------
    wire [N-1:0] u = t[N-1] ? (~t + 1'b1) : t;

    // ---- cabeçalho de 12 bits -------------------------------------------
    wire [11:0] h;
```

```
    generate
        if (N >= 12) begin : g_hdr_wide
            assign h = u[N-1 -: 12];
        end else begin : g_hdr_ghost
            // bits que faltam valem zero
            assign h = {u, {(12-N){1'b0}}};
        end
    endgenerate

    wire [2:0]        r;
    wire signed [8:0] exp_raw;
    takum_hdr_dec u_hdr (.h(h), .r(r), .exp(exp_raw));

    // ---- fração --------- (A fração começa logo depois de S(1) D(1) R(3) C(r): posição 5 + r.) -----
    wire [3:0]   shamt   = 4'd5 + {1'b0, r};  // 5 .. 12
    /* verilator lint_off UNUSEDSIGNAL */  // os 5 bits de baixo nunca são usados
    wire [N-1:0] shifted = u << shamt;        // bits que saem viram zero
    /* verilator lint_on UNUSEDSIGNAL */
    wire [N-6:0] frac_raw = shifted[N-1 -: N-5];

    // ---- saídas ------------------------------------------------------------
    assign sign = special ? 1'b0 : t[N-1];
    assign exp  = special ? 9'sd0 : exp_raw;
    assign frac = special ? {(N-5){1'b0}} : frac_raw;
endmodule
```

Código 4: `rtl/takum_encoder.v` — codificador

```
// takum_encoder.v -- Codificador de takum linear de N bits

`timescale 1ns/1ps

module takum_encoder #(
    parameter N = 16
) (
    input  wire               is_zero,
    input  wire               is_nar,
    input  wire               sign,
    input  wire signed [10:0] exp,
    input  wire [N-4:0]       frac,     // N-3 bits
    input  wire               sticky,
    output wire [N-1:0]       t
);
    localparam [N-1:0] MAXPOS = {1'b0, {(N-1){1'b1}}};   // 0111...1
    localparam [N-1:0] MINPOS = {{(N-1){1'b0}}, 1'b1};   // 0000...1
    localparam [N-1:0] NAR    = {1'b1, {(N-1){1'b0}}};   // 1000...0

    // faixa do expoente -------------------------------------------
    wire too_big   = (exp >  11'sd254);
    wire too_small = (exp < -11'sd255);

    // D, r, R, C --------------------------------------------------------
    wire        d = ~exp[10];                                  // exp >= 0
    /* verilator lint_off UNUSEDSIGNAL */                     // na faixa válida, a cabe em 8 bits
    wire [10:0] a_wide = d ? (exp + 11'sd1) : (-exp);          // 1 .. 255 na faixa
    /* verilator lint_on UNUSEDSIGNAL */
    wire [7:0]  a = a_wide[7:0];
    // priority encoder 8 -> 3: posição do bit 1 mais significativo de a
    reg [2:0] r;
    always @* begin
        casez (a)
            8'b1???????: r = 3'd7;
```

```
            8'b01??????: r = 3'd6;
            8'b001?????: r = 3'd5;
            8'b0001????: r = 3'd4;
            8'b00001???: r = 3'd3;
            8'b000001??: r = 3'd2;
            8'b0000001?: r = 3'd1;
            default:     r = 3'd0;
        endcase
    end

    wire [6:0] mask   = (7'd1 << r) - 7'd1;                 // r bits em 1
    wire [6:0] c_bits = (d ? a[6:0] : ~a[6:0]) & mask;      // C alinhado à direita
    wire [2:0] reg_r  = d ? r : ~r;

    // palavra longa ------------------------------------------------
    // {C (7 bits), frac} << (7 - r) coloca os r bits de C no topo e a fração logo depois,
    qualquer que seja r.
    localparam BW = 7 + (N - 3);                            // largura do corpo
    wire [BW-1:0] body = {c_bits, frac} << (3'd7 - r);

    // full = 0 | D | R(3) | body  -> N + 9 bits
    wire [N+8:0] full = {1'b0, d, reg_r, body};

    // arredondamento (RNE na palavra) ------------------------------
    wire [N-1:0] w     = full[N+8:9];                       // N bits de cima
    wire         guard = full[8];
    wire         stk   = (|full[7:0]) | sticky;
    wire         rnd   = guard & (stk | w[0]);
    wire [N-1:0] w_rnd = w + {{(N-1){1'b0}}, rnd};

    // saturação ----------------------------------------------------
    reg [N-1:0] mag;
    always @* begin
        if (too_big)              mag = MAXPOS;
        else if (too_small)       mag = MINPOS;
        else if (w_rnd[N-1])      mag = MAXPOS;   // "vai-um" chegou ao sinal
        else if (w_rnd == {N{1'b0}}) mag = MINPOS; // nunca vira zero
        else                      mag = w_rnd;
    end

    // sinal e casos especiais --------------------------------------
    wire [N-1:0] signed_word = sign ? (~mag + 1'b1) : mag;

    assign t = is_nar  ? NAR :
               is_zero ? {N{1'b0}} :
                         signed_word;
endmodule
```

Código 5: `rtl/takum_alu.v` — ALU

```
// takum_alu.v - ALU takum (add, sub, mul)

`timescale 1ns/1ps

module takum_alu #(
    parameter N = 16
) (
    input  wire [N-1:0] a,
    input  wire [N-1:0] b,
    input  wire [1:0]   op,
    output wire [N-1:0] y
);
    localparam W  = N - 4;          // largura do significando {1, frac}
    localparam EW = 11;             // largura do expoente que vai ao encoder
```

```
    localparam AW = W + 3;          // significando + guard, round, sticky
    localparam [9:0] AW10 = AW;
    localparam [5:0] AW6  = AW;

    wire [N-1:0] NAR = {1'b1, {(N-1){1'b0}}};

    // ---- decodificação ------------------------------------------------------
    wire              za, na, sa, zb, nb, sb_raw;
    wire signed [8:0] ea, eb;
    wire [N-6:0]      fa, fb;
    takum_decoder #(.N(N)) u_da (.t(a), .is_zero(za), .is_nar(na), .sign(sa),     .exp(ea), .
    frac(fa));
    takum_decoder #(.N(N)) u_db (.t(b), .is_zero(zb), .is_nar(nb), .sign(sb_raw), .exp(eb), .
    frac(fb));

    wire is_mul = (op == 2'd2);
    wire sb     = sb_raw ^ (op == 2'd1);
    wire [W-1:0] ma = {1'b1, fa};
    wire [W-1:0] mb = {1'b1, fb};

    // ---- multiplicação ------------------------------------------------------
    wire [2*W-1:0] prod   = ma * mb;
    wire           p_hi   = prod[2*W-1];          // produto >= 2
    wire [2*W-1:0] p_norm  = p_hi ? prod : (prod << 1);   // bit 1 na posição 2W-1
    wire signed [EW-1:0] m_exp = $signed({{2{ea[8]}}, ea}) + $signed({{2{eb[8]}}, eb}) + (
    p_hi ? 11'sd1 : 11'sd0);
    wire [N-4:0]   m_frac   = p_norm[2*W-2 -: N-3];
    wire           m_stk    = |p_norm[2*W-2-(N-3):0];
    wire           m_sign   = sa ^ sb_raw;

    // ---- soma / subtração ---------------------------------------------------
    wire a_ge_b = ({~ea[8], ea[7:0], fa} >= {~eb[8], eb[7:0], fb});   // expoente com sinal:
    inverte o MSB
    wire signed [8:0] e_big   = a_ge_b ? ea : eb;
    wire signed [8:0] e_small = a_ge_b ? eb : ea;
    wire [W-1:0]      m_big   = a_ge_b ? ma : mb;
    wire [W-1:0]      m_small = a_ge_b ? mb : ma;
    wire              s_big   = a_ge_b ? sa : sb;
    wire              s_small = a_ge_b ? sb : sa;

    wire [9:0] dist = e_big - e_small;
    wire [9:0] dist_c = (dist > AW) ? AW10 : dist;

    wire [AW-1:0]   a_ext  = {m_big, 3'b000};
    wire [AW-1:0]   b_ext  = {m_small, 3'b000};
    wire [2*AW-1:0] b_wide = {b_ext, {AW{1'b0}}} >> dist_c;
    wire [AW-1:0]   b_al   = b_wide[2*AW-1:AW];
    wire            b_stk  = |b_wide[AW-1:0];               // bits que saíram do
    alinhamento
    wire [AW-1:0]   b_jam  = {b_al[AW-1:1], b_al[0] | b_stk};

    wire            sub    = s_big ^ s_small;
    wire [AW:0]     sum    = sub ? ({1'b0, a_ext} - {1'b0, b_jam}) : ({1'b0, a_ext} + {1'b0,
    b_jam});

    // contador de zeros à esquerda
    reg [5:0] lz;
    integer i;
    always @* begin
        lz = AW6 + 6'd1;                            // sum == 0
        for (i = 0; i <= AW; i = i + 1)
            if (sum[i]) lz = AW6 - i[5:0];
    end
```

```
    wire [AW:0]   s_norm  = sum << lz;              // bit 1 em AW
    wire signed [EW-1:0] a_exp = $signed({{2{e_big[8]}}, e_big}) + 11'sd1 - $signed({5'b0, lz
    });
    wire [N-4:0]  a_frac  = s_norm[AW-1 -: N-3];
    wire          a_stk   = |s_norm[AW-1-(N-3):0];
    wire          a_zero  = (sum == 0);

    // ---- encoder ------------------------------------------------------------------
    wire              enc_zero = is_mul ? 1'b0 : a_zero;
    wire              enc_sign = is_mul ? m_sign : s_big;
    wire signed [EW-1:0] enc_exp = is_mul ? m_exp : a_exp;
    wire [N-4:0]      enc_frac = is_mul ? m_frac : a_frac;
    wire              enc_stk  = is_mul ? m_stk  : a_stk;
    wire [N-1:0]      y_core;
    takum_encoder #(.N(N)) u_enc (.is_zero(enc_zero), .is_nar(1'b0), .sign(enc_sign),
                                  .exp(enc_exp), .frac(enc_frac), .sticky(enc_stk), .t(y_core
    ));

    // ---- casos especiais
    ----------------------------------------------------------------
    wire [N-1:0] b_signed = (op == 2'd1) ? (~b + 1'b1) : b;      // -b, em complemento de 2
    assign y = (na | nb)  ? NAR :
               is_mul     ? ((za | zb) ? {N{1'b0}} : y_core) :
               za         ? (zb ? {N{1'b0}} : b_signed) :
               zb         ? a :
                            y_core;
endmodule
```

Código 6: `rtl/takum_vec_alu.v` — unidade vetorial

```
// takum_vec_alu.v -- Unidade vetorial Zvtakum: vtadd.vv, vtsub.vv, vtmul.vv

`timescale 1ns/1ps

module takum_vec_alu #(
    parameter VLEN = 64
) (
    input  wire [VLEN-1:0] vs2,
    input  wire [VLEN-1:0] vs1,
    input  wire [VLEN-1:0] vd_old,
    input  wire [1:0]      op,
    input  wire [1:0]      sew,
    input  wire [7:0]      vl,        // nº de elementos ativos
    output wire [VLEN-1:0] vd
);
    wire [VLEN-1:0] r8, r16, r32;

    genvar i;
    generate
        for (i = 0; i < VLEN/8; i = i + 1) begin : g8
            takum_alu #(.N(8)) u (.a(vs2[8*i +: 8]), .b(vs1[8*i +: 8]), .op(op), .y(r8[8*i +:
     8]));
        end
        for (i = 0; i < VLEN/16; i = i + 1) begin : g16
            takum_alu #(.N(16)) u (.a(vs2[16*i +: 16]), .b(vs1[16*i +: 16]), .op(op), .y(r16
    [16*i +: 16]));
        end
        for (i = 0; i < VLEN/32; i = i + 1) begin : g32
            takum_alu #(.N(32)) u (.a(vs2[32*i +: 32]), .b(vs1[32*i +: 32]), .op(op), .y(r32
    [32*i +: 32]));
        end
    endgenerate

    wire [VLEN-1:0] res = (sew == 2'd0) ? r8 : (sew == 2'd1) ? r16 : r32;
```

```
32
33      // máscara de elementos ativos: i < vl
34      reg [VLEN-1:0] act;
35      integer k, w;
36      always @* begin
37          w = (sew == 2'd0) ? 8 : (sew == 2'd1) ? 16 : 32;
38          for (k = 0; k < VLEN; k = k + 1)
39              act[k] = ((k / w) < vl);
40      end
41
42      assign vd = (res & act) | (vd_old & ~act);
43  endmodule
```